UNIVERSIDAD DIEGO PORTALES

FACULTAD DE INGENIERÍA Y CIENCIAS



Tarea 2: Protocolo IRC mediante contenedores

Estudiantes:

Diego Maldonado

Nicolas Quintanilla

November 10, 2026

Índice

1 Introducción **1**

2 Desarrollo **1**

2.1 Capítulo 1: Marco teórico 1

2.2 Capítulo 2: Despliegue y análisis experimental 2

3 Conclusiones **5**

A Anexos **5**

A.1 Figuras y Tablas 5

Referencias **12**

Enlaces de la tarea **12**

1 Introducción

Internet Relay Chat (IRC) es un protocolo de comunicación en tiempo real basado en el intercambio de mensajes entre clientes y servidores. Su formato de texto facilita observar e interpretar los comandos de protocolo durante una sesión, por lo que resulta adecuado para estudiar el comportamiento del tráfico de red en un entorno controlado. En este proyecto se prepara una arquitectura contenerizada con InspIRCd como servidor IRC y WeeChat como cliente, y se analiza el tráfico generado y capturado durante escenarios definidos.

El objetivo general es evaluar el protocolo IRC mediante contenedores, comparando el tráfico esperado según las especificaciones con el observado en las capturas. También se plantea una hipótesis experimental sobre la posible alteración de mensajes en tránsito (on-the-fly) y se identifican las condiciones y limitaciones necesarias para comprobarla.

El documento se organiza de la siguiente manera: el Capítulo 1 presenta el marco teórico y el comportamiento de tráfico esperado; el Capítulo 2 describe el despliegue, la captura y el análisis experimental; las conclusiones contrastan expectativas y observaciones; y las referencias y anexos reúnen las fuentes, figuras y tablas utilizadas.

2 Desarrollo

2.1 Capítulo 1: Marco teórico

2.1.1 Protocolo IRC

IRC define la comunicación entre clientes y servidores mediante mensajes de texto estructurados en comandos y parámetros. RFC 1459 describe el protocolo original, mientras que RFC 2812 especifica el protocolo de cliente y detalla comandos y respuestas numéricas utilizados en la interacción con el servidor [1, 2]. Estas RFC constituyen la referencia para establecer el comportamiento esperado; las extensiones implementadas por las herramientas pueden añadir mensajes que deben identificarse por separado.

El propósito de IRC es facilitar la teleconferencia multiusuario en tiempo real mediante mensajes de texto sobre una arquitectura distribuida. Su origen se sitúa en 1988, cuando se implementó para que usuarios conectados a un sistema de tablón de anuncios (BBS) conversaran entre sí; por ello, es más preciso describirlo como una herramienta de chat inicialmente asociada a un BBS que como una alternativa general a estos sistemas. El protocolo admite comunicación privada entre dos usuarios (uno a uno) y comunicación grupal (uno a muchos) a través de canales temáticos. Su formato textual basado en líneas mantiene el intercambio sencillo, mientras que los servidores retransmiten mensajes y mantienen el estado de clientes y canales necesario para la sesión; IRC no define persistencia del historial de conversación [1, 2].

Una sesión típica comprende la conexión al servidor, el registro del nickname y del usuario, la recepción de respuestas de bienvenida, la entrada a un canal y el intercambio de mensajes. Durante la sesión también pueden aparecer comandos de mantenimiento de conexión (por ejemplo, `PING` y `PONG`) y, al terminar, un comando de desconexión. Los nombres, parámetros, respuestas y orden observados se documentarán y contrastarán con las RFC; no se asumirán extensiones o capacidades que no hayan sido verificadas en la configuración ensayada.

2.1.2 Tráfico esperado

Para este ensayo se utiliza deliberadamente una conexión sin TLS, en el puerto estándar `6667` y con el parámetro `-notls` en el cliente, para inspeccionar directamente la carga útil en texto plano. Se registra y analiza la siguiente secuencia esperada:

1. **Establecimiento TCP:** saludo de tres vías (`SYN`, `SYN-ACK` y `ACK`) entre cliente y servidor.
2. **Registro:** envío en texto claro de `NICK` y `USER` desde WeeChat para definir el apodo y los datos de usuario.
3. **Bienvenida:** respuestas numéricas `001`–`004` (`RPL_WELCOME`, `RPL_YOURHOST`, `RPL_CREATED` y `RPL_MYINFO`), seguidas del mensaje del día (MOTD).
4. **Ingreso al canal:** solicitud `JOIN` a `#redes` y recepción de la lista de participantes.
5. **Intercambio:** envío de mensajes con `PRIVMSG` al canal, delimitados por retorno de carro y salto de línea (CR-LF).
6. **Mantenimiento:** intercambio de mensajes `PING` y `PONG` para comprobar la disponibilidad de los extremos.
7. **Cierre:** comando `QUIT` y terminación de la conexión TCP, normalmente mediante `FIN/ACK`; también puede observarse un `RST` al cerrarse el socket.

La correspondencia entre la secuencia prevista, los mensajes involucrados y sus especificaciones según RFC 2812 se detalla en la Tabla 2 del Anexo B.

2.1.3 Contenedores y alcance del ensayo

Docker permite ejecutar los componentes en entornos aislados y reproducibles. En esta práctica, InspIRCd prestó el servicio IRC y WeeChat generó el tráfico de cliente. La topología, las imágenes y etiquetas utilizadas, la red y los parámetros de captura se resumen en la Tabla 3 del Anexo.

2.2 Capítulo 2: Despliegue y análisis experimental

2.2.1 Despliegue de InspIRCd y WeeChat en Docker

El despliegue se realizó con InspIRCd como servidor IRC y WeeChat como cliente en contenedores separados, conectados a la red privada `red-irc` en modo puente. Las imágenes, etiquetas utilizadas

y parámetros del entorno se registran en la Tabla 3 del Anexo. La documentación oficial de InspIRCd, WeeChat y Docker se utilizó como referencia [3, 4, 5].

1. Se construyó `servidor/Dockerfile` a partir de la imagen `inspircd/inspircd-docker:latest` (etiqueta utilizada: `latest`), exponiendo el puerto `6667`.
2. Se empaquetó el cliente en `cliente/Dockerfile` sobre `alpine:latest` (etiqueta utilizada: `latest`), instalando el binario `weechat` mediante APK.
3. La orquestación y la conexión a la red privada `red-irc` se definieron en `docker-compose.yml`; los servicios se iniciaron simultáneamente con `docker compose up -d`.
4. La interacción se realizó con `docker exec -it cliente-irc weechat --dir /tmp/weechat`; en WeeChat se configuró `/server add local servidor-irc/6667 -notls` y se inició la conexión con `/connect local`.

2.2.2 Captura y análisis del tráfico

La captura se realizó en la máquina anfitriona Kali Linux con Wireshark, sobre la interfaz virtual tipo puente `br-73de0fc6d41b`, creada por Docker Compose para la red `red-irc`, y no sobre la interfaz física `eth0`. Se aplicó el filtro de visualización `tcp.port == 6667` para aislar el flujo entre el cliente WeeChat (`172.18.0.3`, puerto efímero `49224`) y el servidor InspIRCd (`172.18.0.2`, puerto `6667`). La captura `capturas/trafico_irc.pcapng` se almacenó en el repositorio para mantener la trazabilidad de la evidencia.

La secuencia temporal muestra que el ingreso al canal comenzó en el paquete `40` con la solicitud Request (`JOIN`). En el paquete `41`, el servidor respondió con las características de la sala y etiquetas cronológicas `@time`. Entre los paquetes `43` y `51` se observaron comandos de sincronización del canal (`MODE` y `WHO`), intercalados con acuses de recibo TCP convencionales.

El mensaje de prueba se transmitió en el paquete `52` (Request `PRIVMSG`). La inspección hexadecimal de la carga útil TCP confirmó la transmisión íntegra en ASCII de "`PRIVMSG #redes :Holaaa probando comunicacion IRC con WeeChat e InspIRC`", con una longitud de `72` bytes y terminación en `0x0D 0x0A`, correspondientes a retorno de carro y salto de línea según RFC 2812 [2]. En el paquete `53`, el servidor acusó recibo y replicó el mensaje al canal mediante la extensión `inspircd.org/echo`.

El cierre de la sesión comenzó en el paquete `69` con Request `QUIT`. El servidor procesó la salida en el paquete `70` e inició la terminación del transporte con la bandera `FIN` en el paquete `72`. A continuación, la pila TCP del contenedor cliente emitió un segmento `RST, ACK` (paquete `73`), completando el cierre observado de la conexión. La topología y la captura se referencian en las Figuras 1 y 7; la Tabla 4 resume los hitos y la comparación entre el tráfico esperado y el observado. Además, la Tabla 2 presenta los diez tipos de tráfico observados durante el ciclo de vida de la sesión IRC.

2.2.3 Hipótesis de alteración *on-the-fly*

Al no utilizar TLS en el puerto `6667`, los mensajes IRC viajan en texto plano y podrían ser leídos o modificados por un intermediario con acceso al flujo de comunicación. A partir de esta condición, se plantean cinco posibles escenarios de alteración del tráfico y sus eventuales repercusiones sobre InspIRCd y WeeChat.

Tabla 1: Posibles efectos de alterar el tráfico IRC en tránsito.

Modificación del tráfico	Posible repercusión en InspIRCd	Posible repercusión en WeeChat
Alterar `JOIN`: cambiar el nombre del canal solicitado.	El servidor podría procesar la solicitud con un canal distinto al solicitado originalmente. Si este no existe o no cumple las condiciones de acceso, podría rechazar la unión.	El usuario podría ingresar a otro canal o recibir un error de unión, sin entrar al canal que pretendía.
Modificar el contenido de `PRIVMSG`: reemplazar el texto del mensaje.	El servidor podría retransmitir al canal el contenido adulterado, ya que el protocolo IRC no incorpora por sí mismo mecanismos de integridad o firmas de mensajes.	Si WeeChat muestra el texto original mediante eco local, el emisor podría visualizarlo mientras los demás usuarios reciben el contenido modificado, afectando la coherencia de la conversación.
Modificar el destinatario de `PRIVMSG`: cambiar el usuario o canal al que va dirigido el mensaje.	El servidor podría enrutar el mensaje hacia un destinatario diferente del original, siempre que este sea válido.	El mensaje podría no llegar al destinatario previsto y, en su lugar, ser recibido por otro usuario o canal.
Alterar los delimitadores CR-LF (`0x0D 0x0A`).	El servidor podría mantener los datos en el búfer de entrada hasta recibir el fin de línea. Si la línea supera el máximo de 512 octetos, incluidos los delimitadores, podría rechazar el comando o cerrar la conexión, según la implementación.	El cliente podría experimentar retrasos, falta de respuesta del servidor o pérdida de sincronización aparente de la conversación.
Suprimir respuestas `PONG`.	Si no recibe la respuesta esperada al `PING`, el servidor podría agotar el temporizador de actividad y cerrar la conexión con *Ping timeout*.	WeeChat perdería la sesión de forma abrupta, mostraría una alerta de desconexión y podría intentar reconectarse si esa opción está habilitada.

De estos cinco escenarios, la supresión de respuestas `PONG` fue verificada experimentalmente en el laboratorio: las capturas muestran el cierre del enlace por *Ping timeout* (Tabla 4). La Figura 2 ilustra este comportamiento mediante una comparación entre el intercambio normal de mensajes `PING`/`PONG` y el escenario en que la respuesta `PONG` es suprimida, provocando que InspIRCd agote el temporizador de actividad y cierre la conexión. Los otros cuatro escenarios —alteración de `JOIN`, modificación del contenido y destinatario de `PRIVMSG`, y alteración de los delimitadores CR-LF— corresponden a hipótesis técnicas basadas en el formato y los comandos descritos por RFC 2812. Sus efectos concretos dependen de la implementación y no se comprobaron en este ensayo [2].

3 Conclusiones

El desarrollo de esta experiencia permitió desplegar con éxito una arquitectura cliente-servidor para IRC mediante contenedores Docker independientes, orquestados sobre una red privada virtual y utilizando InspIRCd y WeeChat. La captura desde la interfaz virtual del anfitrión permitió inspeccionar el flujo entre ambos contenedores.

La inspección con Wireshark constató, en las tramas analizadas, concordancia entre el comportamiento observado y el formato descrito por RFC 2812: los mensajes `JOIN`, `PRIVMSG` y `QUIT` se transmitieron como texto ASCII en claro, delimitados por CR-LF. También se observaron extensiones IRCv3, como las etiquetas temporales de los mensajes [2].

Al estar deshabilitado TLS en el puerto `6667`, el contenido de la sesión quedó visible en la captura y, en principio, expuesto a lectura o alteración por un intermediario con acceso al flujo. La intervención controlada de supresión de respuestas `PONG` produjo un *Ping timeout* y la desconexión del cliente; en cambio, la modificación de delimitadores u otros comandos no se ensayó directamente y se considera un riesgo, no un resultado observado. El ensayo se circunscribió a una red local controlada, sin interacción con redes externas y sin TLS, condición adoptada para los fines académicos de esta experiencia.

A Anexos

A.1 Figuras y Tablas

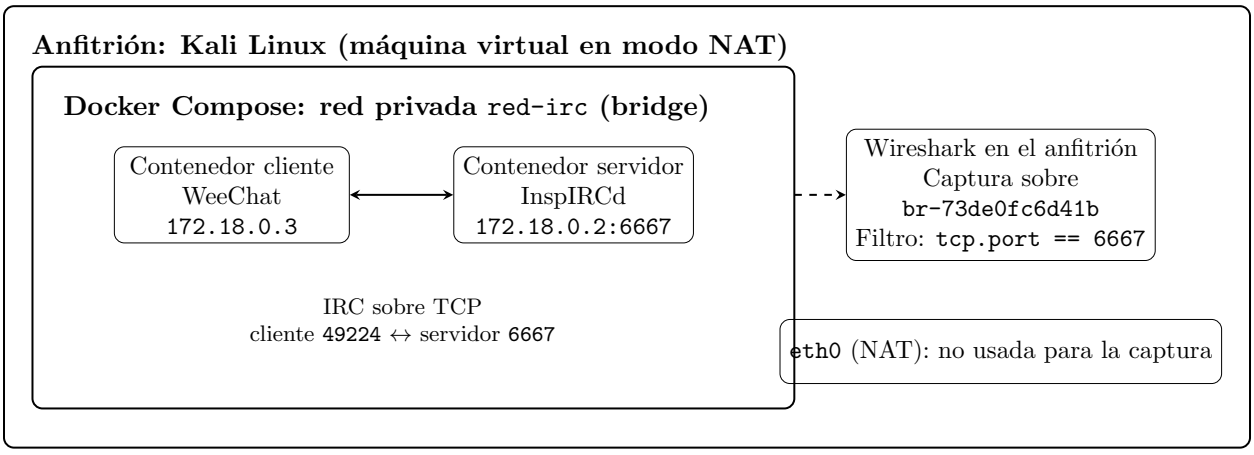


Figure 1: Topología del entorno experimental. WeeChat e InspIRCd se comunican por la red puente privada `red-irc`; Wireshark captura sobre `br-73de0fc6d41b`, no sobre `eth0`.

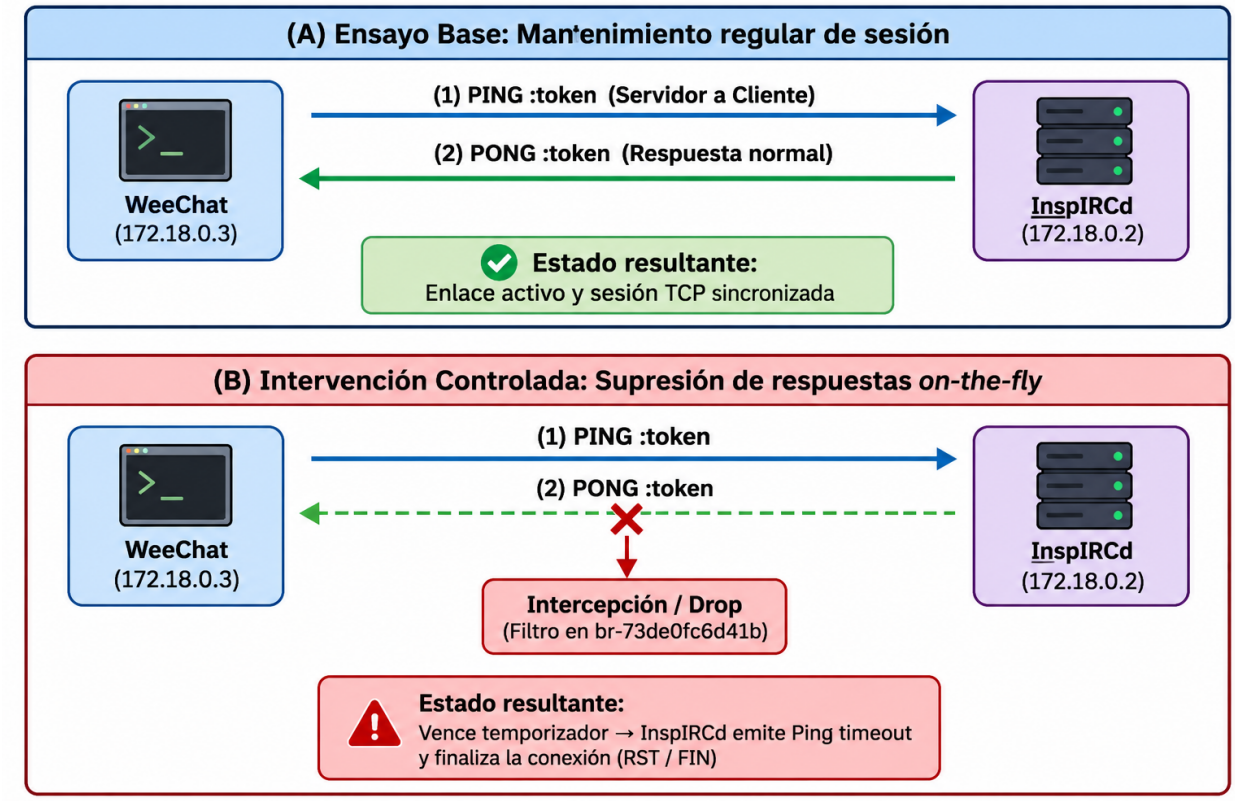


Figure 2: Comparación del intercambio normal de mensajes PING/PONG con la supresión en tránsito de las respuestas PONG y el consiguiente Ping timeout.



Figure 3: Docker muestra el proceso de compilación de las imágenes a partir de los Dockerfiles.

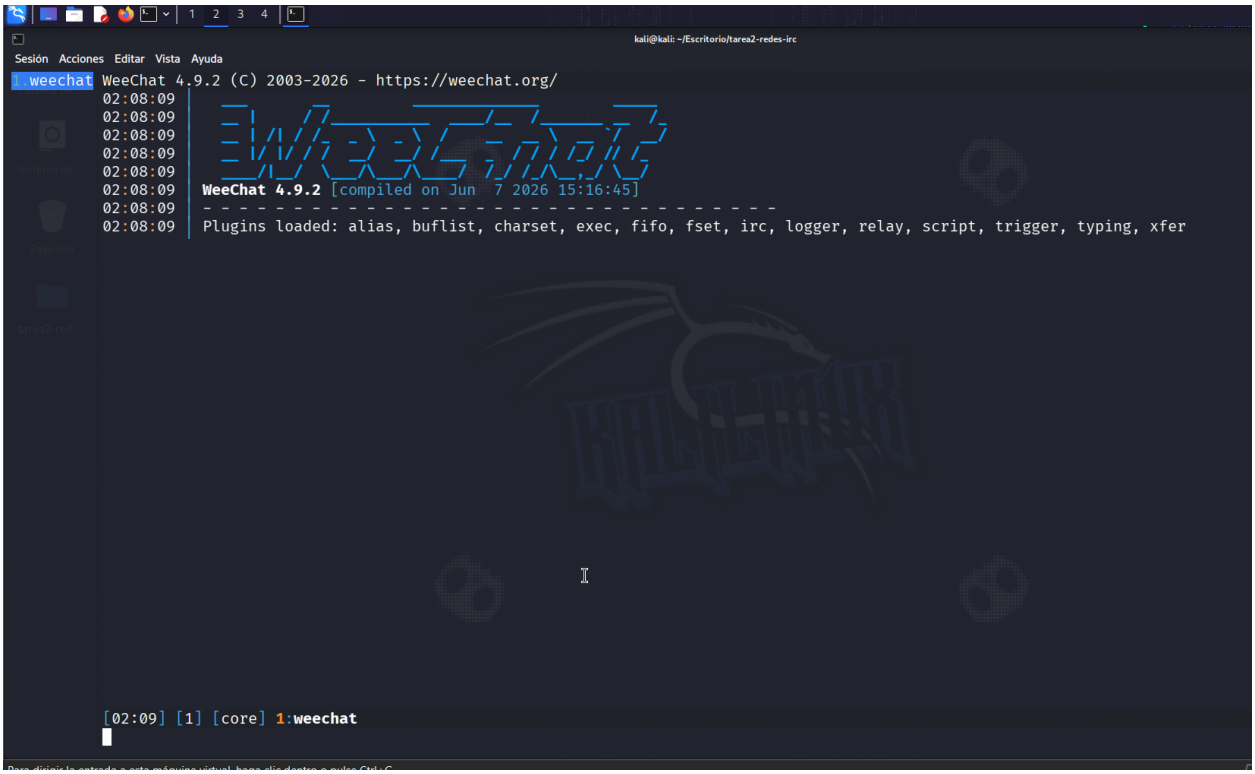


Figure 4: Ejecución de WeeChat en el contenedor cliente.

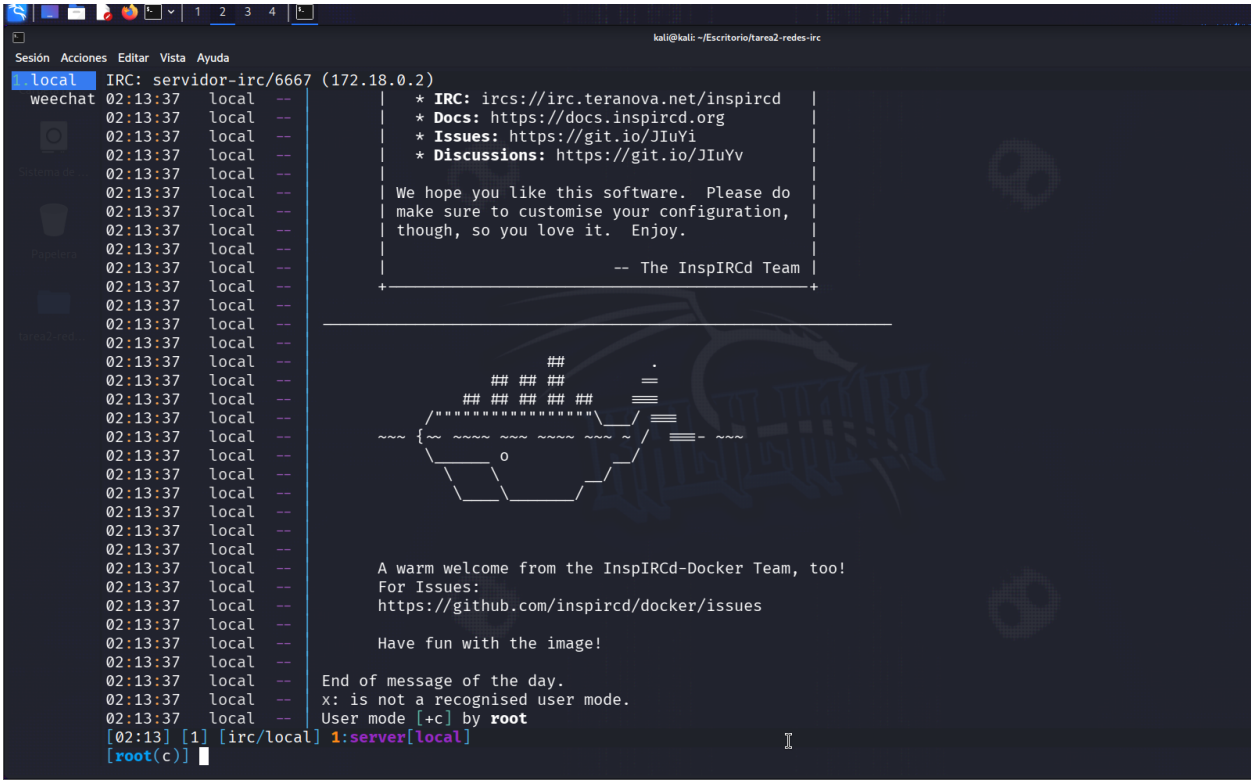


Figure 5: WeeChat conectado al servidor InspIRCd en el contenedor.

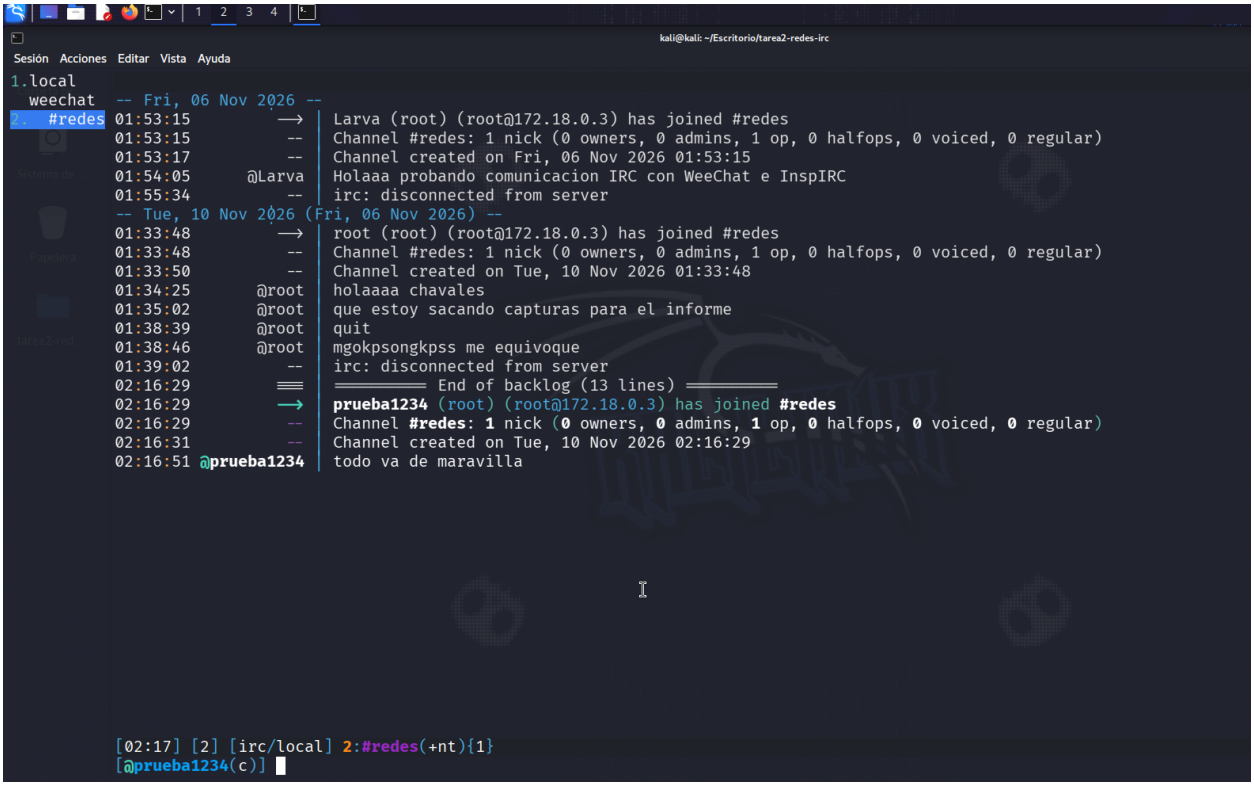


Figure 6: Intercambio de mensajes IRC en el canal `#redes` mediante WeeChat.

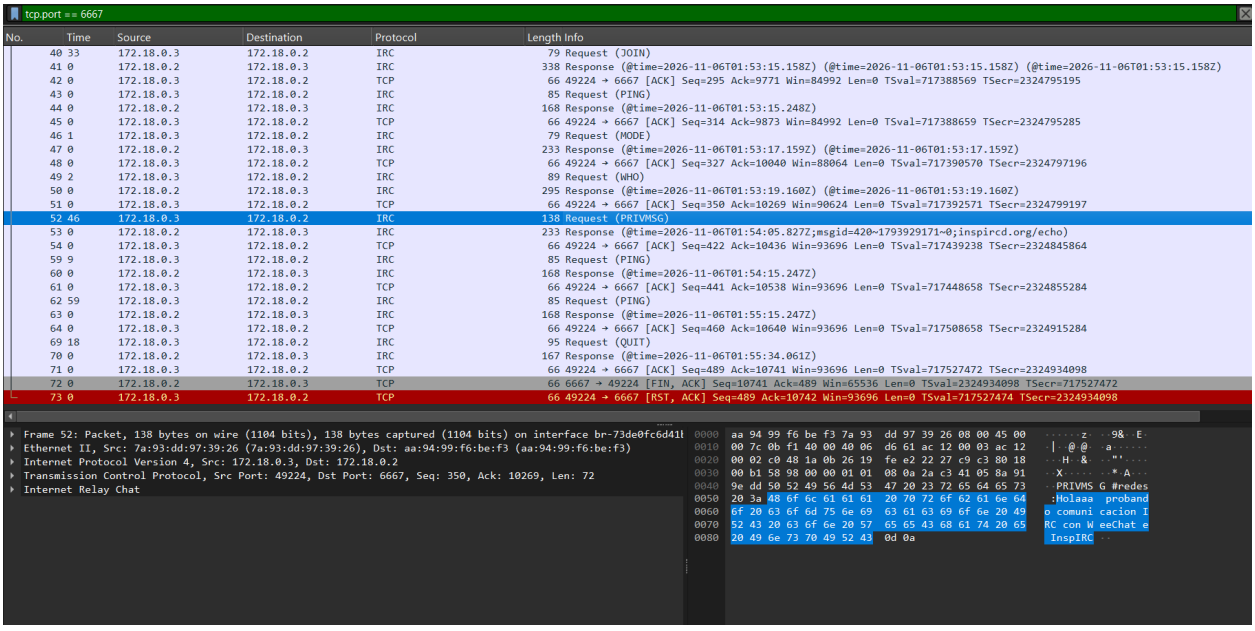


Figure 7: Captura de tráfico IRC en Wireshark sobre la interfaz virtual `br-73de0fc6d41b` con el filtro `tcp.port == 6667`. Se aprecia el flujo completo entre WeeChat (`172.18.0.3`) e InspIRCd (`172.18.0.2`), destacando en el paquete `52` el envío del comando `PRIVMSG` con el texto del mensaje en claro y su terminación en los bytes hexadecimales `0d 0a`.

Table 2: Secuencia y caracterización de los 10 tipos de tráfico observados en el ciclo de vida de la sesión IRC.

Tipo / Fase	Cliente a servidor (WeeChat → InspIRCd)	Servidor a cliente (InspIRCd → WeeChat)	Evidencia en captura
1. TCP Handshake	Envío de segmento con bandera `SYN` (puerto `49224` a `6667`).	Respuesta con banderas `SYN, ACK`; el cliente finaliza con `ACK`.	Previo al paquete `40`
2. Registro NICK / USER	Emisión de comandos en texto plano `NICK` y `USER`.	Procesamiento y reserva de apodo en memoria.	Previo al paquete `40`
3. Bienvenida y capacidades	En espera de confirmación de registro.	Ráfaga numérica `001` a `004`, mensaje del día (MOTD) y etiquetas `@time`.	Previo al paquete `40`
4. Ingreso a canal (JOIN)	Solicitud explícita `JOIN #redes`.	Creación del canal y envío de estados `353` (`RPL_NAMREPLY`) y `366`.	Paquetes `40` y `41`
5. Sincronización de modos (MODE)	Comando `MODE #redes` para auditar permisos del canal.	Respuesta con modos asignados al operador (`+nt`).	Paquetes `46` y `47`
6. Consulta de miembros (WHO)	Consulta `WHO #redes` para listar información extendida.	Códigos numéricos `352` (`RPL_WHOREPLY`) y `315` (`RPL_ENDOFWHO`).	Paquetes `49` y `50`
7. Mensajería (PRIVMSG)	Transmisión de mensaje en texto claro delimitado por `0x0D 0x0A`.	Acuse de recibo y eco hacia los suscriptores (`inspircd.org/echo`).	Paquetes `52` y `53`
8. Mantenimiento Keep-Alive	Acuse de presencia inmediato con trama `PONG :<token>`.	Consulta periódica de enlace con trama `PING :<token>`.	Paquetes `43--44` y `59--60`
9. Salida de aplicación (QUIT)	Instrucción `QUIT :<motivo>` solicitando el término de la sesión.	Confirmación de desuscripción de canales y registro de salida.	Paquetes `69` y `70`
10. Cierre de Transporte TCP	Recepción de cierre y remisión de segmento con bandera `RST, ACK`.	Inicio de cierre de socket mediante segmento con bandera `FIN, ACK`.	Paquetes `72` y `73`

Table 3: Registro de versiones y configuración del entorno.

Elemento	Valor registrado
Sistema anfitrión / Docker	Kali Linux (máquina virtual en modo NAT) / Docker Engine 28.5.2 (docker –version)
Imagen y etiqueta utilizada de InspIRCd	`inspircd/inspircd-docker:latest` (etiqueta `latest`; no identifica una versión concreta)
Imagen base y etiqueta utilizada para WeeChat	`alpine:latest` (etiqueta `latest`; no identifica una versión concreta), con el paquete `weechat` instalado vía APK
Red, puerto y modo de conexión	Red virtual personalizada `red-irc` (bridge de Docker); puerto `6667/TCP`
TLS y parámetros relevantes	Deshabilitado; cliente WeeChat ejecutado con `-notls`, transmisión en texto plano
Interfaz, filtro y archivo de captura	`br-73de0fc6d41b` (`ip -brief link`); filtro `tcp.port == 6667`; archivo `capturas/trafico_irc.pcapng`

Table 4: Comparación del tráfico esperado con el observado.

Ensayo / fase	Esperado	Observado (paquete/tiempo)	Diferencia y evidencia
Base: registro	Handshake TCP de tres vías (`SYN`, `SYN-ACK`, `ACK`), registro de usuario con `NICK` y `USER`, seguido de respuestas numéricas de bienvenida (`001`–`004`) y el mensaje del día (`MOTD`) del servidor.	Paquetes TCP de inicio y ráfaga inicial de registro. En las tramas previas al paquete `40` se negocian los datos de sesión entre `172.18.0.3` y `172.18.0.2` en el puerto `6667`.	El comportamiento observado es consistente con la secuencia de conexión y registro descrita en RFC 2812. Además, se identificaron extensiones IRCv3, como las etiquetas `@time` incorporadas en los mensajes del servidor.
Base: canal y mensaje	Solicitud `JOIN` para ingresar a una sala, validación con listado de integrantes (`RPL_NAMREPLY`) y envío del mensaje en texto claro mediante `PRIVMSG` hacia el canal.	Paquete `40`: solicitud JOIN a las `01:53:15`; paquete `41`: respuesta con usuarios y tema; paquete `49`: WHO; paquete `52`: solicitud PRIVMSG a las `01:54:05`, con carga útil de `72` bytes.	No se observó discrepancia con el estándar. El visor hexadecimal del paquete `52` muestra la cadena "`PRIVMSG #redes :`Holaaa probando comunicacion IRC con WeeChat e InspIRC", terminada estrictamente en los bytes `0x0D 0x0A` (retorno de carro y salto de línea).
Base: cierre	Emisión del comando `QUIT` por el cliente, acuse de recibo del servidor y desconexión formal del transporte TCP mediante intercambio de banderas `FIN` y `ACK`.	Paquete `69`: solicitud QUIT a las `01:55:34`; paquete `70`: respuesta del servidor; paquete `72`: cierre FIN, ACK del servidor; paquete `73`: RST, ACK emitido por el cliente a las `01:55:34`.	La terminación a nivel de aplicación fue limpia. Tras el FIN del servidor, la captura muestra que el cliente emitió un segmento RST, ACK; se describe la secuencia observada sin atribuirle una liberación específica de recursos.
Intervención controlada	Comportamiento anómalo ante la pérdida de sincronización de presencia mediante los comandos de mantenimiento `PING` y `PONG`.	Se identificó tráfico recurrente de sondeo en los paquetes `43--44`, `59--60` y `62--63`, donde cliente y servidor intercambian solicitudes PING y respuestas PONG en intervalos regulares.	Al suprimir en tránsito las respuestas PONG, el servidor InspIRCd agotó su temporizador de inactividad (*Ping timeout*), cerró unilateralmente el enlace y desconectó al usuario.

References

[1] J. Oikarinen y D. Reed, "Internet Relay Chat Protocol," RFC 1459, Internet Engineering Task Force, mayo de 1993. https://www.rfc-editor.org/rfc/rfc1459.

[2] C. Kalt, "Internet Relay Chat: Client Protocol," RFC 2812, Internet Engineering Task Force, abril de 2000. https://www.rfc-editor.org/rfc/rfc2812.

[3] InspIRCd, "InspIRCd Documentation." https://docs.inspircd.org/.

[4] WeeChat, "WeeChat User's Guide." https://weechat.org/files/doc/stable/weechat_user.en.html.

[5] Docker, "Docker documentation." https://docs.docker.com/.

[6] Wireshark Foundation, "Wireshark User's Guide." https://www.wireshark.org/docs/wsug_html_chunked/.

Enlaces de la tarea

- **Repositorio de GitHub:** https://github.com/Panxxto/tarea2-redes-irc
- **Video de YouTube:** https://youtu.be/0QAO_j9GIl4